

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



SANCTUM: ABILITIES E REVOGAÇÃO DE TOKENS

Autorização de acesso à API com o Middleware Abilities, Revogação e Configuração de Expiração de Tokens.

Prof. Gill Velleda Gonzales
gillgonzales@ifsul.edu.br | [@gillgonzales](#) | [@g1ll](#)
CSTSI – IV Semestre – Desenvolvimento Back-End II
Novembro - 2025

API Token com Laravel Sanctum

Tópicos da Aula



- **Habilidades com o Sanctum**
 - Conceito Principal
 - Alteração da migration de User
 - Implementando Habilidades
 - Expiração de Tokens
 - Revogação de Tokens
- **Atividades**
- **Referências**

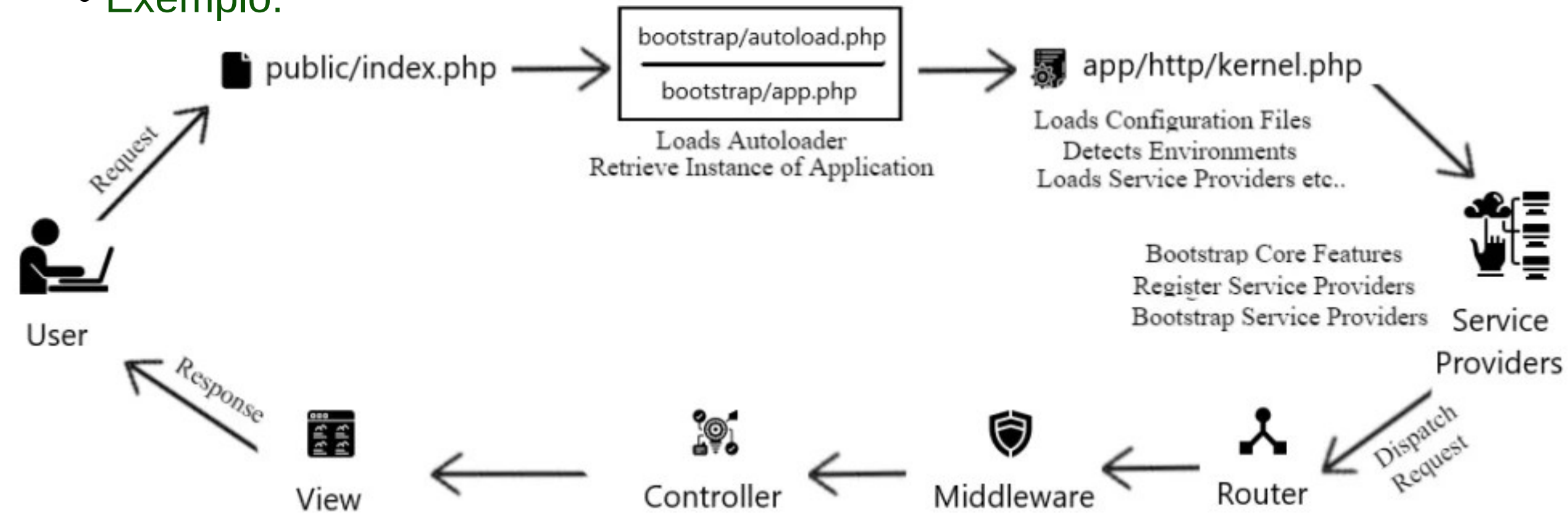
Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

- **Request Lifecycle**

- **Exemplo:**



FONTE: <https://dev.to/patelparixit07/laravel-request-lifecycle-195e>

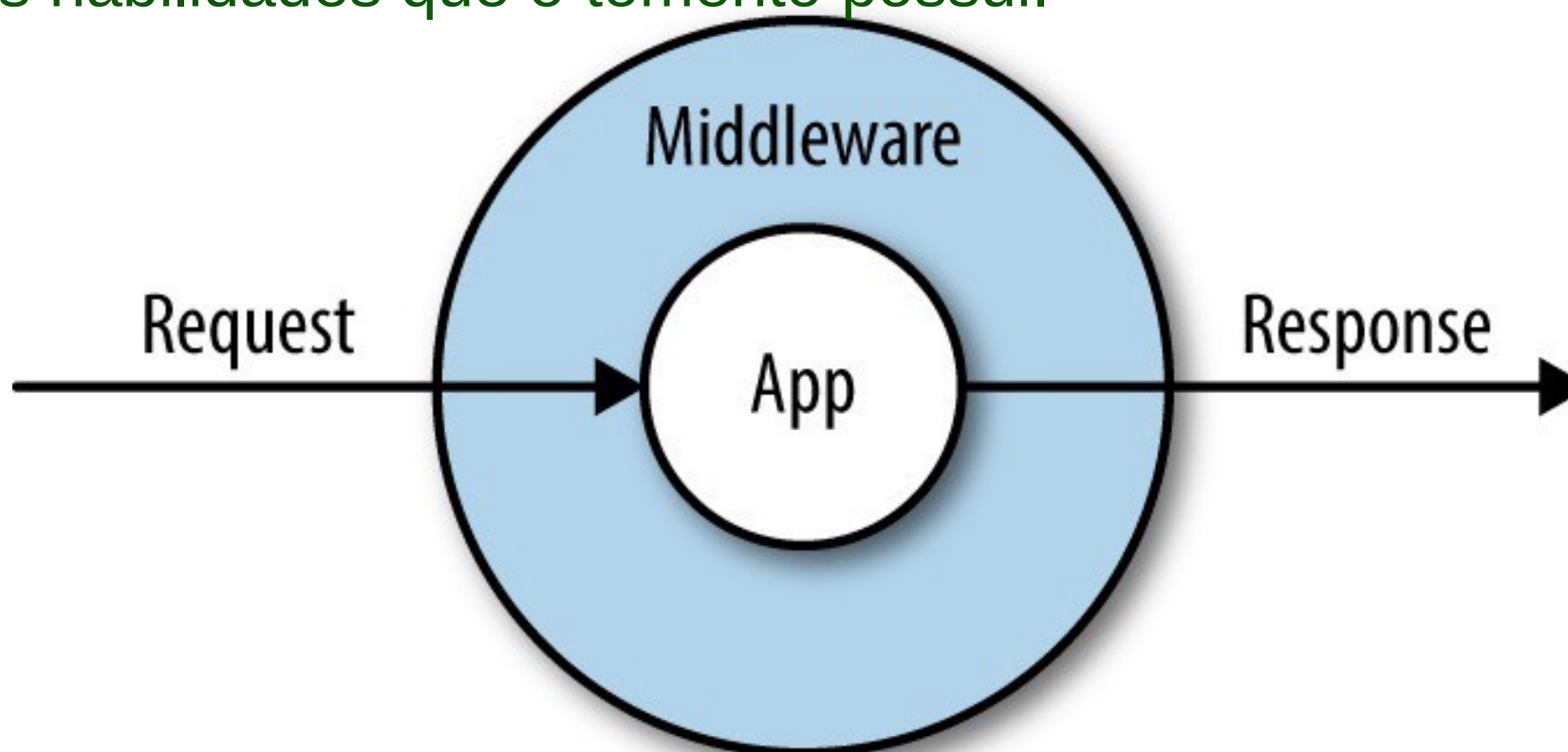
Middlewares e Request Lifecycle do Laravel

Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel



- **Middleware**

- São os mecanismos de inspeção e filtro de requisições HTTP;
- A **autorização** de acesso é feita por um middleware, responsável por verificar as habilidades que o usuário possui.
- Exemplo:



FONTE: <https://tony-stark.medium.com/laravel-5-8-middleware-tutorial-with-example-2019-9f5069b8c3cc>

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



Conceito:

- As habilidades (***abilities***) que o ***Laravel Sanctum*** implementa são uma simplificação de atribuição de escopos aos ***tokens*** gerados.
- Podemos ter diferentes tipos de habilidades para diferentes usuários;
- O ***token*** de um usuário com permissões de ***administrador*** deverá ser diferente do ***token*** de um usuário ***comum***.
- Portanto, são as habilidades (***abilities***) que o token do Sanctum implementa que poderão fazer esta diferenciação.
- Podemos encontrar a coluna de habilidades na tabela de tokens do sanctum: ***personal_access_tokens***

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



Conceito:

- Tabela ***personal_access_tokens*** com a coluna ***abilities***

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	id 	bigint		UNSIGNED	No	None
☐	2	tokenable_type 	varchar(255)	utf8mb4_unicode_ci		No	None
☐	3	tokenable_id 	bigint		UNSIGNED	No	None
☐	4	name	varchar(255)	utf8mb4_unicode_ci		No	None
☐	5	token 	varchar(64)	utf8mb4_unicode_ci		No	None
☐	6	abilities	text	utf8mb4_unicode_ci		Yes	NULL
☐	7	last_used_at	timestamp			Yes	NULL
☐	8	created_at	timestamp			Yes	NULL
☐	9	updated_at	timestamp			Yes	NULL

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



Conceito:

- As habilidades são cadastradas no ***momento da criação*** do ***token***.
- Desta forma, podemos depois verificar quais habilidades aquele ***token*** possui e **prover** ou **negar** o acesso.

CRIAÇÃO DURANTE O LOGIN

```
return $user->createToken('token-name', ['server:update'])->plainTextToken;
```

```
if ($user->tokenCan('server:update')) {  
    //  
}
```

USO EM ALGUM
CONTROLADOR

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



• Alteração da Migration de User

Vamos criar uma habilidade relacionada a um novo atributo no modelo de Usuário (**User**), este atributo será o **is_admin**.

Para armazenarmos este novo atributo vamos alterar a **migration** da tabela de usuário e recriar o banco de dados.

```
database > migrations > 2014_10_12_000000_create_users_table.php > PHP Intelephense >

14 public function up()
15 {
16     Schema::create('users', function (Blueprint $table) {
17         $table->id();
18         $table->string('name');
19         $table->string('email')->unique();
20         $table->timestamp('email_verified_at')->nullable();
21         $table->string('password');
22         $table->boolean('is_admin')->default(false);
23         $table->rememberToken();
24         $table->timestamps();
25     });
26 }
```


Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Alteração da Migration de User**

database > migrations > 2014_10_12_000000_create_users_table.php > PHP Intelephense >

```
14 public function up()
15 {
16     Schema::create('users', function (Blueprint $table) {
17         $table->id();
18         $table->string('name');
19         $table->string('email')->unique();
20         $table->timestamp('email_verified_at')->nullable();
21         $table->string('password');
22         $table->boolean('is_admin')->default(false);
23         $table->rememberToken();
24         $table->timestamps();
25     });
26 }
```

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Atualização do DatabaseSeeder**

- Adicionamos ao método run da classe DatabaseSeeder a criação de três usuários comuns e um admin do sistema padrão.
- A senha padrão é salva de forma criptografada pelo Hash::make.

```
aula_05 > database > seeders > DatabaseSeeder.php > PHP Intelephense > DatabaseSeeder > run
17 public function run(): void
18 {
19     \App\Models\User::factory(3)→create();
20
21     \App\Models\User::factory()→create([
22         'name'      ⇒ 'Admin User',
23         'email'     ⇒ 'admin@sysadmin.test',
24         'is_admin'  ⇒ true,
25         'password'  ⇒ Hash::make('4dm1n@Sy$123')
26     ]);
27
```

You, 2 minutes ago • Uncommitted changes

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Atualização do DatabaseSeeder**

- Podemos refatorar o código anterior para ler a senha padrão a partir do **env**.

```
aula_05 > database > seeders > DatabaseSeeder.php > PHP Intelephense > DatabaseSeeder > run
18     public function run(): void
19     {
20         \App\Models\User::factory(3)→create();
21
22         $admin_pass = env('APP_ADMIN_PASS');
23         if(empty($admin_pass))
24             throw new Exception("ERRO: Admin Password!");
25
26         \App\Models\User::factory()→create([
27             'name'      ⇒ 'Admin User',
28             'email'     ⇒ 'admin@sysadmin.test',
29             'is_admin'  ⇒ true,
30             'password'  ⇒ Hash::make($admin_pass)
31         ]);
```

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Atualização do DatabaseSeeder**

- Caso não seja definida a variável **APP_ADMIN_PASS** no **env** uma exceção é lançada na hora de rodar o comando do **artisan**.

```
2023_05_01_200500_create_media_table.php:11: warning: This file will be overwritten by Laravel's migrations installer. Please backup it before continuing.

INFO Seeding database.

Exception

ERRO: Admin Password! Defina a variável APP_ADMIN_PASS.

at database/seeder/DatabaseSeeder.php:24
20 |         \App\Models\User::factory(3)->create();
21 |
```

- Agora é preciso criar a variável **APP_ADMIN_PASS** no **env** para popular o banco e gerar o **admin** padrão.

```
aula_05 > .env
5 APP_URL=http://localhost:8001
6 APP_ADMIN_PASS=4dm1n@Sy$123
```

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Migration e Seeder**

- Vamos executar as migrations com o artisan e atualizar o banco de dados.

php artisan migrate:fresh

```
echoes@echoes-Inspiron-15-3567 > ...Projetos/tsi_daoo/introLaravel > <?php 8.1.4 > sanctum-abilities
$ php artisan migrate:fresh
Dropped all tables successfully.
Migration table created successfully.
Migrating: 2014_10_12_000000_create_users_table
Migrated: 2014_10_12_000000_create_users_table (376.94ms)
Migrating: 2014_10_12_100000_create_password_resets_table
Migrated: 2014_10_12_100000_create_password_resets_table (152.93ms)
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (347.77ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (645.85ms)
Migrating: 2022_03_08_193201_create_fornecedores_table
Migrated: 2022_03_08_193201_create_fornecedores_table (91.24ms)
Migrating: 2022_03_08_200000_create_produtos_table
Migrated: 2022_03_08_200000_create_produtos_table (733.30ms)
```

- Agora podemos popular o banco com o seed: **php artisan db:seed**
- Ou rodar de só uma vez o equivalente: **php artisan migrate:fresh --seed**

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- Refatorando com UserAdminSeeder

- Podemos refatorar o código e criar a classe **UserAdminSeeder**, passando a lógica de criação para o seu método **run**.

- **php artisan make:seeder UserAdminSeeder**

```
aula_05 > database > seeders > UserAdminSeeder.php > PHP Intelephense > UserAdminSeeder
9  class UserAdminSeeder extends Seeder
10 {
11     public function run(): void
12     {
13         $admin_pass = env('APP_ADMIN_PASS');
14         if(empty($admin_pass))
15             throw new \Exception("ERRO: Admin Password! Defina a variável APP_ADMIN_PASS.");
16
17         \App\Models\User::factory()->create([
18             'name'      => 'Admin User',
19             'email'      => 'admin@sysadmin.test',
20             'is_admin'   => true,
21             'password'   => Hash::make($admin_pass)
22         ]);
23
```


Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- Refatorando DatabaseSeeder

- Então atualizamos a classe **DatabaseSeeder** para apenas executar o método **run** da nova classe **UserAdminSeeder**.

```
aula_05 > database > seeders > DatabaseSeeder.php > PHP Intelephense > DatabaseSeeder > run
10  class DatabaseSeeder extends Seeder
11  {
12      /**
13       * Seed the application's database.
14       */
15      public function run(): void
16      {
17          $this->call([
18              UserAdminSeeder::class,
19              RegiaoSeeder::class,
20              EstadoSeeder::class,
21              PromocaoSeeder::class
22          ]);
```

- Recriamos o banco: **php artisan migrate:fresh --seed**

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- Criação da Habilidade

- No momento do login vamos criar a nova **habilidade** para o token, como segundo parâmetro do método **createToken**, baseada na coluna **is_admin**:

aula_05 > app > Http > Controllers > Api > Auth > LoginController.php > PHP Intelephense > LoginController > login > \$ability

```
13 public function login(LoginRequest $request)
14 {
15     try{
16         $user = User::where('email',$request->email)->first();
17         if(!$user || !Hash::check($request->password,$user->password))
18             throw new \Exception('Falha na autenticação!');
19
20         $ability = $user->is_admin?['is-admin']:[];
21         $token = $user->createToken($request->email,$ability)->plainTextToken;
22         return response()->json(['token'=>$token]);
23     }catch(\Exception $error){
```

- Agora usaremos esta habilidade em alguma ação do sistema, neste exemplo usaremos no método **store** da classe **Fornecedor**.

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Verificação da Habilidade no Controlador**

- Agora poderemos verificar se existe a **habilidade** no token de autenticação da requisição.
- Usamos o método **tokenCan** a partir do usuário autenticado e passamos por parâmetro o nome da habilidade (**is-admin**);

aula_05 > app > Http > Controllers > Api > FornecedorController.php > PHP Intelephense > FornecedorController > store

```
24 public function store(Request $request)
25 {
26     $statusHttp = 500;
27     try {
28         if(!$request->user()->tokenCan('is-admin')){
29             $statusHttp = 403;//Forbidden - Sem permissão
30             throw new \Exception('Você não tem permissão!');
31         }
32         $updatedFornecedor = $request->all();
33         $storedFornecedor = Fornecedor::create($updatedFornecedor);
34         return response()->json([
```

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Verificação da Habilidade através de Middlewares**

- Poderemos verificar as habilidades através de ***middlewares*** e diretamente nas rotas;
- Existem dois middlewares possíveis: ***abilities*** e ***ability***
 - ***CheckAbilities***: Verifica se todas as habilidades necessárias estão registradas no token. Neste caso deverá possuir todas as habilidades para ter acesso.
 - ***CheckForAnyAbility***: Verifica se pelo menos uma das habilidades necessárias existe no token enviado na requisição. Neste caso o token não precisa implementar todas as habilidades, e poderá ter acesso ao método com apenas uma habilidade.

/app/Http/Kernel.php

```
'abilities' => \Laravel\Sanctum\Http\Middleware\CheckAbilities::class,  
'ability' => \Laravel\Sanctum\Http\Middleware\CheckForAnyAbility::class,
```


Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Verificação da Habilidade através de Middlewares:**

- Exemplos de uso de *abilities* e *ability*

- **AND:**

```
Route::get('/orders', function () {  
    // Token has both "check-status" and "place-orders" abilities...  
})->middleware(['auth:sanctum', 'abilities:check-status,place-orders']);
```

- **OR:**

```
Route::get('/orders', function () {  
    // Token has the "check-status" or "place-orders" ability...  
})->middleware(['auth:sanctum', 'ability:check-status,place-orders']);
```

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Ativação dos Middlewares na Inicialização**

- Para usar os middlewares deveremos ativá-los na inicialização, localizado no arquivo ***bootstrap/app.php*** e habilitar o ***\$middleware->alias[...]***.

```
1  use Laravel\Sanctum\Http\Middleware\CheckAbilities;
2  use Laravel\Sanctum\Http\Middleware\CheckForAnyAbility;
3
4  →withMiddleware(function (Middleware $middleware) {
5      $middleware->alias([
6          'abilities' => CheckAbilities::class,
7          'ability' => CheckForAnyAbility::class,
8      ]);
9  })
```


Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- **Utilização dos Middlewares nas Rotas**

- Com o *middleware* devidamente ativado, poderemos utilizá-lo nas rotas.
- Neste exemplo apenas o token com a habilidade *is-admin* poderá acessar o métodos *post*, *put*, *patch* e *delete* da classe Fornecedor.

```
aula_05 > routes > api.php > Closure
20
27 Route::middleware('auth:sanctum')->group(function () {
28
29     Route::apiResource('fornecedores', FornecedorController::class)
30         ->parameters(['fornecedores'=>'fornecedor'])
31         ->middleware('ability:is-admin');
32
33     Route::controller(FornecedorController::class)->group(function(){
34         Route::get('fornecedores', 'index');
35         Route::get('fornecedores/{fornecedor}', 'show');
36         Route::get('fornecedores/{fornecedor}/produtos', 'produtos');
37         Route::get('fornecedores/regiao/{nomeRegiao}', 'regiao');
38     });
39
```

- As rotas do tipo *get* podem ser acessadas com o *token comum* apenas.

Habilidades com Sanctum

Introdução ao conceito de Habilidades no Sanctum



- Utilização dos Middlewares nas Rotas
 - Continuidade da declaração de *rotas agrupadas (laravel tips)*:

```
aula_05 > routes >  api.php > ...
```

```
39
40     Route::apiResource('produtos', ProdutoController::class);
41     Route::apiResource('users', UserController::class);
42     Route::post('logout', [LoginController::class, 'logout']);
43 }); //grupo do sanctum
44
45 Route::controller(ProdutoController::class)→group(function(){
46     Route::get('produtos', 'index');
47     Route::get('produtos/{produto}', 'show');
48 });
49
50 Route::post('users', [UserController::class, 'store']);
51 Route::post('login', [LoginController::class, 'login']);
52
```

- Exemplo do arquivo completo neste [link](#).



- **FUNÇÕES PARA REVOGAÇÃO DE TOKENS**

Existem duas formas de se revogar os tokens gerados para um determinado usuário:

- ***tokens()* → *delete()*:**

A função ***tokens()*** provém acesso a todos os tokens de um determinado usuário, combinado com o método ***delete()*** removemos todos os seus tokens.

- ***currentAccessToken()* → *delete()*:**

Neste caso, com a função ***currentAccessToken()*** teremos acesso apenas ao token utilizado durante a requisição, e poderemos revogar apenas este token com o método ***delete()***.

Tempo de Expiração de um Tokens

Configuração do tempo de expiração de tokens no Sanctum



- **Expiração:**

- Por padrão os tokens do Sanctum nunca expiram e só podem ser destruídos pela revogação.
- No entanto, podemos configurar o Sanctum para expirar os tokens e determinar um tempo limite.
- No arquivo de configuração ***config/sanctum.php*** encontramos a variável ***expiration*** a qual podemos definir um valor em **MINUTOS**. O seu valor padrão é ***null***.

```
config >  sanctum.php
45      */
46
47      'expiration' => 525600,
48
49      /*
```



DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009.

Laravel Tips. disponível em:
<https://laravel-news.com/laravel-route-organization-tips>, acesso em 04/12/2023.

Laravel Docs – **Middleware**, disponível em:
<https://laravel.com/docs/master/middleware>, acesso em 26/10/2025.

Laravel Docs – **Request Lifecycle**, disponível em:
<https://laravel.com/docs/master/lifecycle>, acesso em 26/10/2025.

Laravel Docs – **Laravel Sanctum**, disponível em:
<https://laravel.com/docs/master/sanctum>, acesso em 25/10/2025.